Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

Observability

University of Minho

2025-2026

Observability

What it is and Why it Matters

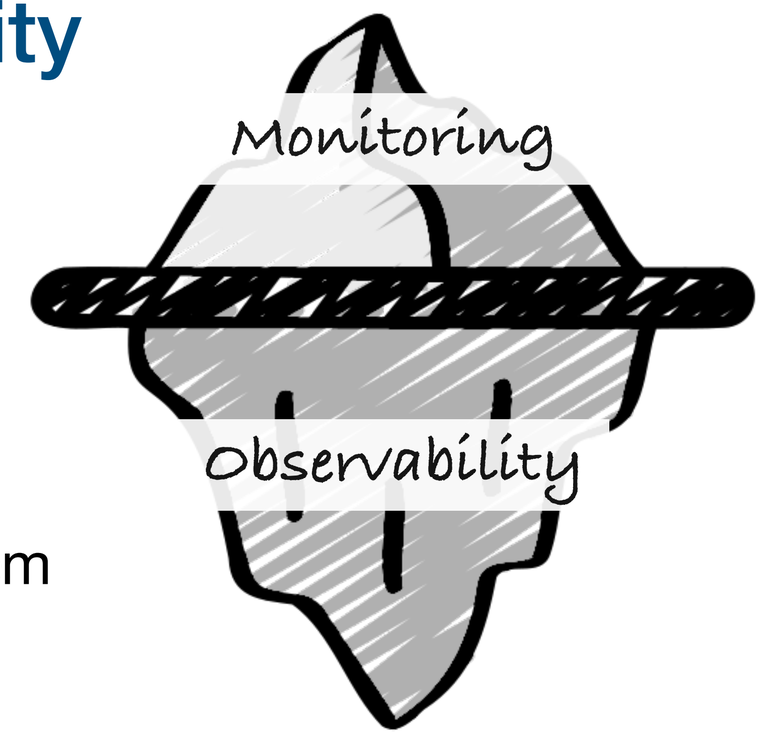
- **Observability** provides insight into how systems and applications behave, helping operators and developers understand both high- and low-level activity.

- **Utility for OS developers and users:**
  - Gain visibility into OS internals (e.g., scheduling, caching, memory, I/O).
  - Understand how applications interact with the OS and resources (e.g., syscalls, CPU/memory usage, network I/O).

- **Use Cases**:
  - Enhance Visibility — Know what's happening inside your system at all layers.
  - Root Cause Analysis — Quickly identify why issues occur.
  - Optimized Performance — Detect bottlenecks and improve resource utilization.
  - Compliance & Auditing — Track system behavior and maintain audit trails.

Observability

Monitoring vs Observability



- **Monitoring** *(the tip of the iceberg)*
  - Detect and alert issues
  - Tells ***that*** something is wrong
- **Observability** *(the part below the surface)*
  - Investigate and understand the root cause of the problem
  - Helps figure out ***why*** it is wrong
    - *Why* did it happen?
    - *How* did it happen?
    - *When* did it happen?
    - *Where* did it happen?

Observability Purposes

Debugging

- **Debugging** — to identify, isolate, and fix problems.
  - **Ad Hoc:** inspect code by inserting temporary output statements.
    - Examples: Add print() statements around suspicious code, print variable values.
  - **Logging**: record program events.
    - By using **logging libraries**: zlog (C), Log4j (Java), logging module (Python), …
    - Or **Third-Party & System Logs:** database/web server logs, journalctl (system logs), dmesg (kernel logs), …
  - **Debuggers**: use tools to monitor, pause, and control the program execution.
    - Examples: gdb (C/C++), jdb (Java), pdb (Python).
  - **Tracers:** use tools to track operations performed by a program.
    - Examples: strace (syscalls), ltrace (library calls), ftrace (kernel function calls).

Observability Purposes

Profiling

- **Profiling** — to understand which parts of a program consume the most time and/or resources.
  - **Ad Hoc**: modify code to measure execution time between two points.
    - Example: Printing timestamps before and after a function call.
  - **Profilers**: use tools to systematically measure program behavior.
    - Record function calls or periodically sample the program state (e.g., every millisecond) and aggregate statistics to show where time or resources are spent.
    - Examples:
      - CPU profilers: gprof (C/C++), cProfile (Python)
      - Memory profilers: Valgrind (C/C++), memory_profiler (Python)
      - Event profilers: Perf (CPU cycles, cache misses, branch predictions)

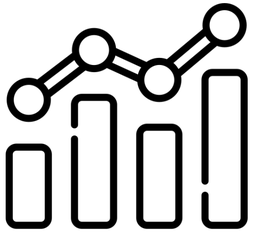
Observability Purposes

Resource Monitoring

- **<u>Resource monitoring</u>** — to understand the resource consumption of a program or system in real-time.
  - **Process & System Overview**: monitor processes and overall system health.
    - Examples: top/htop (interactive process viewers), ps (process snapshot).
  - **CPU & Memory**: track CPU utilization and memory consumption.
    - Examples: mpstat (CPU stats), free (memory usage), vmstat (system stats).
  - **Disk & I/O**: monitor disk activity, space usage, and open files.
    - Examples: iotop (I/O by process), df/du (disk space), lsof (open files).
  - **Network**: inspect connections, interfaces, and bandwidth usage.
    - Examples: ss/netstat (connections), ifconfig/ip (interfaces), nethogs/iftop (traffic).

Observability Data

The Three Pillars



Metrics

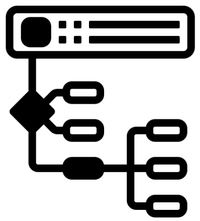
The ***What*** of Observability

+

Logs

The ***Why*** of Observability

+



Traces

The ***How*** and ***Where*** of Observability

Observability Data

The Three Pillars

- **Metrics** — numeric representation of data measured that reflect the system's performance and health over time.
  - Examples: Memory/disk/CPU usage, latency, throughput, error rate.
  - Helpful in understanding what is happening in the system.
    - What resources are overloaded?
    - Is latency increasing?

Observability Data

The Three Pillars

- **Logs** — timestamped records of events occurring within a system.
  - Can be plaintext, binary, or structured (e.g., JSON format).
  - Examples: User logins, error messages, service status.
  - Helpful in understanding why an issue happened.
    - Why did the login fail?
    - Why did the database service not start?

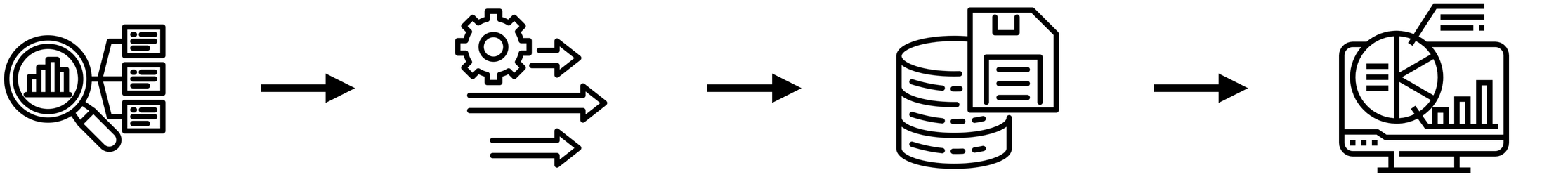
Observability Data

The Three Pillars

- **Traces** — special type of logs that record how requests flow through a system.
  - Can be local (e.g., system calls issued by a program) or distributed (e.g., a request traversing multiple components in a distributed system).
  - Helpful in understanding *where* and *how* a problem occurred.
    - Which sequence of operations led to the crash?
    - Which service introduced the failure?
    - How did this request propagate through the system?

Observability Flow

Subtitle Text

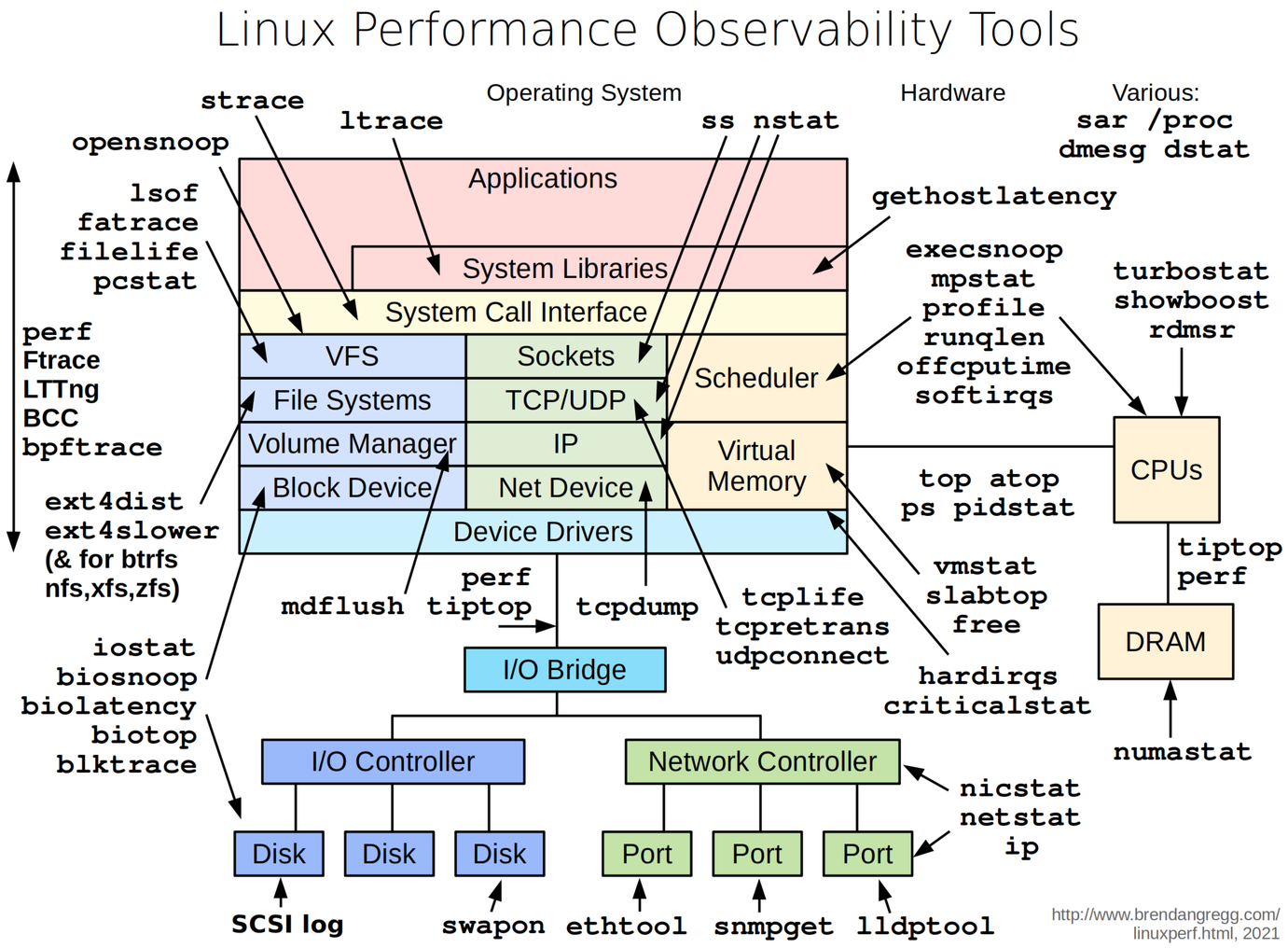


Data Collection → **Data Transport** → **Data Storage** → **Data Analysis & Visualization**

Data Collection

Where to collect

Observability data can be collected from multiple points across the system.

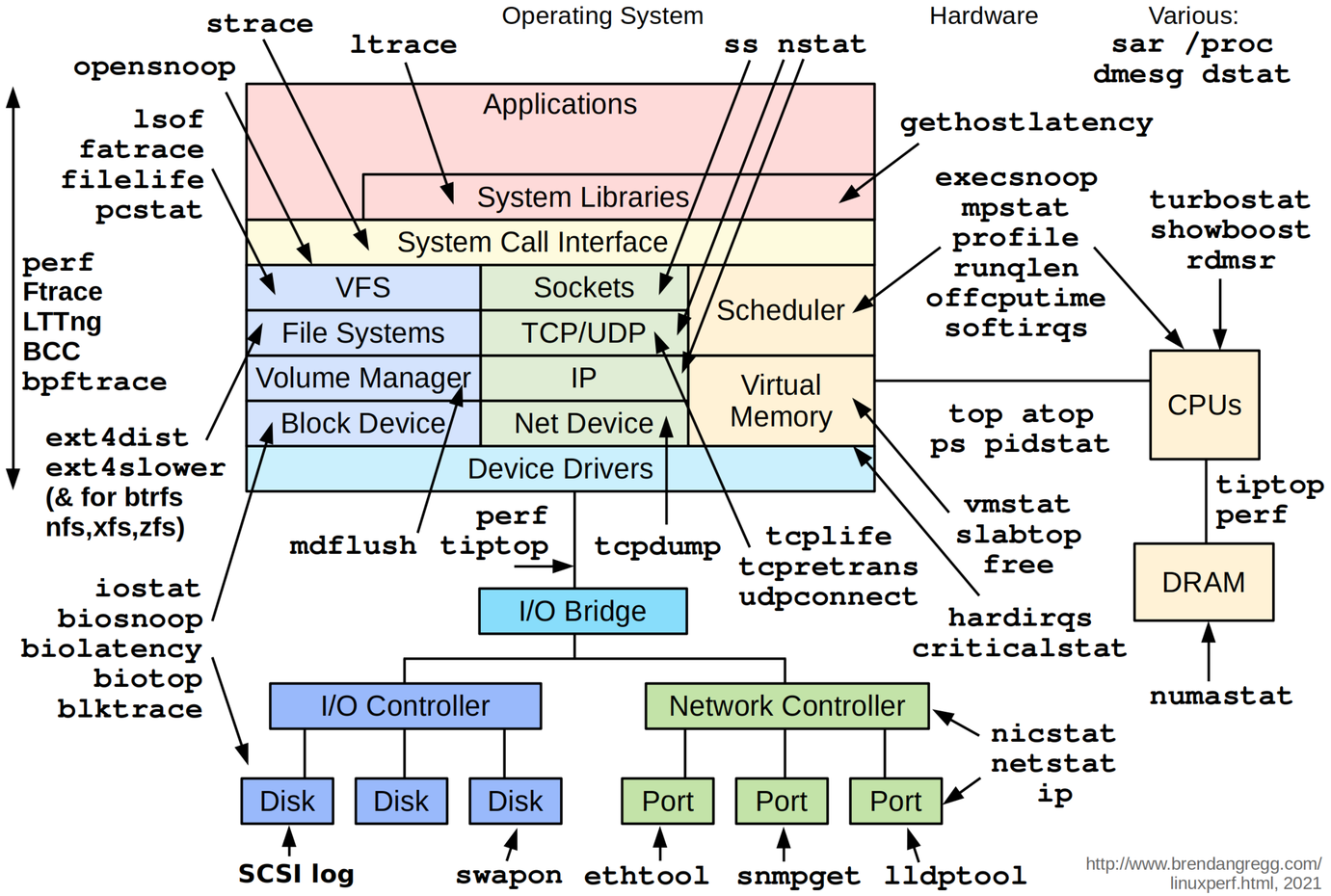


Data Collection

Where to collect

- At the **application** level
  - Captures user-facing logic and business operations.
  - Example: business logic, request handlers, internal APIs.
  - Tools: zlog/Log4j (logging libraries), Jaeger/ Zipkin/ OpenTelemetry (distributed tracing), …

Linux Performance Observability Tools

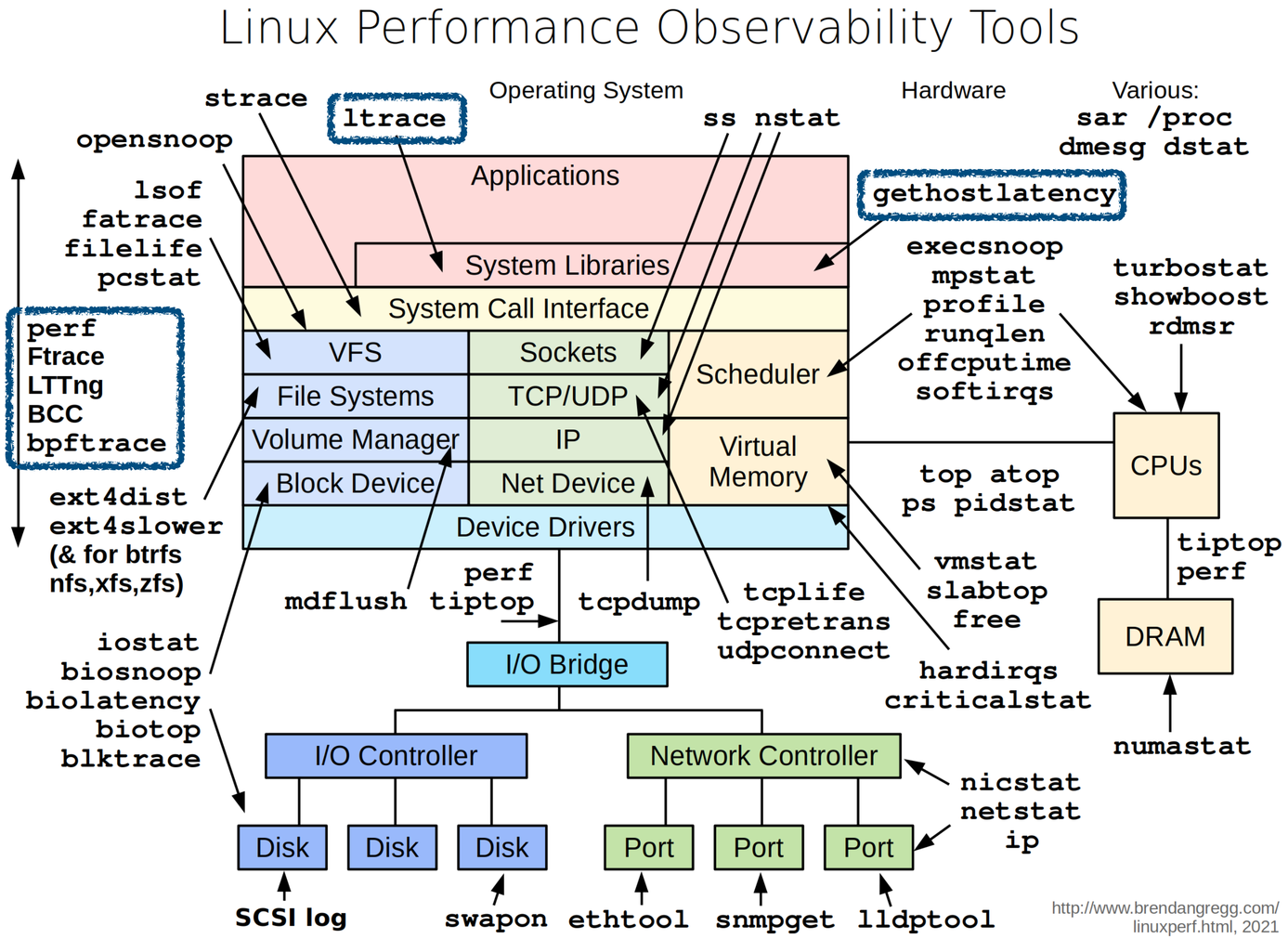


http://www.brendangregg.com/linuxperf.html, 2021

Data Collection

Where to collect

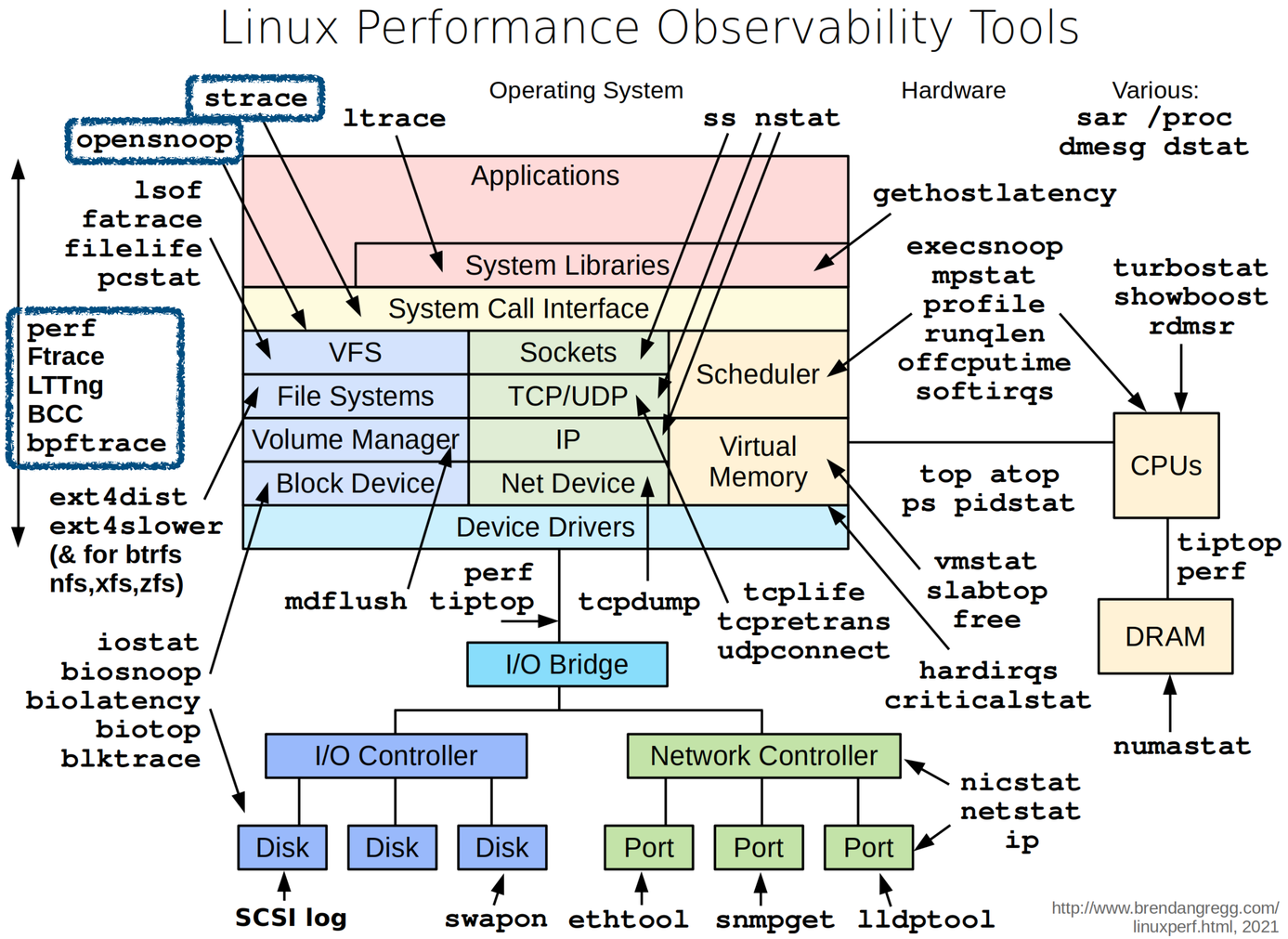
- At the **system libraries** level
  - Captures behavior of shared runtime components used by applications.
  - Example: malloc()/free() (*libc*), pthread_create()/ pthread_join() (*libpthread*), crypt()/md5()(*libcrypt*).
  - Tools: ltrace (library calls), …



Data Collection

Where to collect

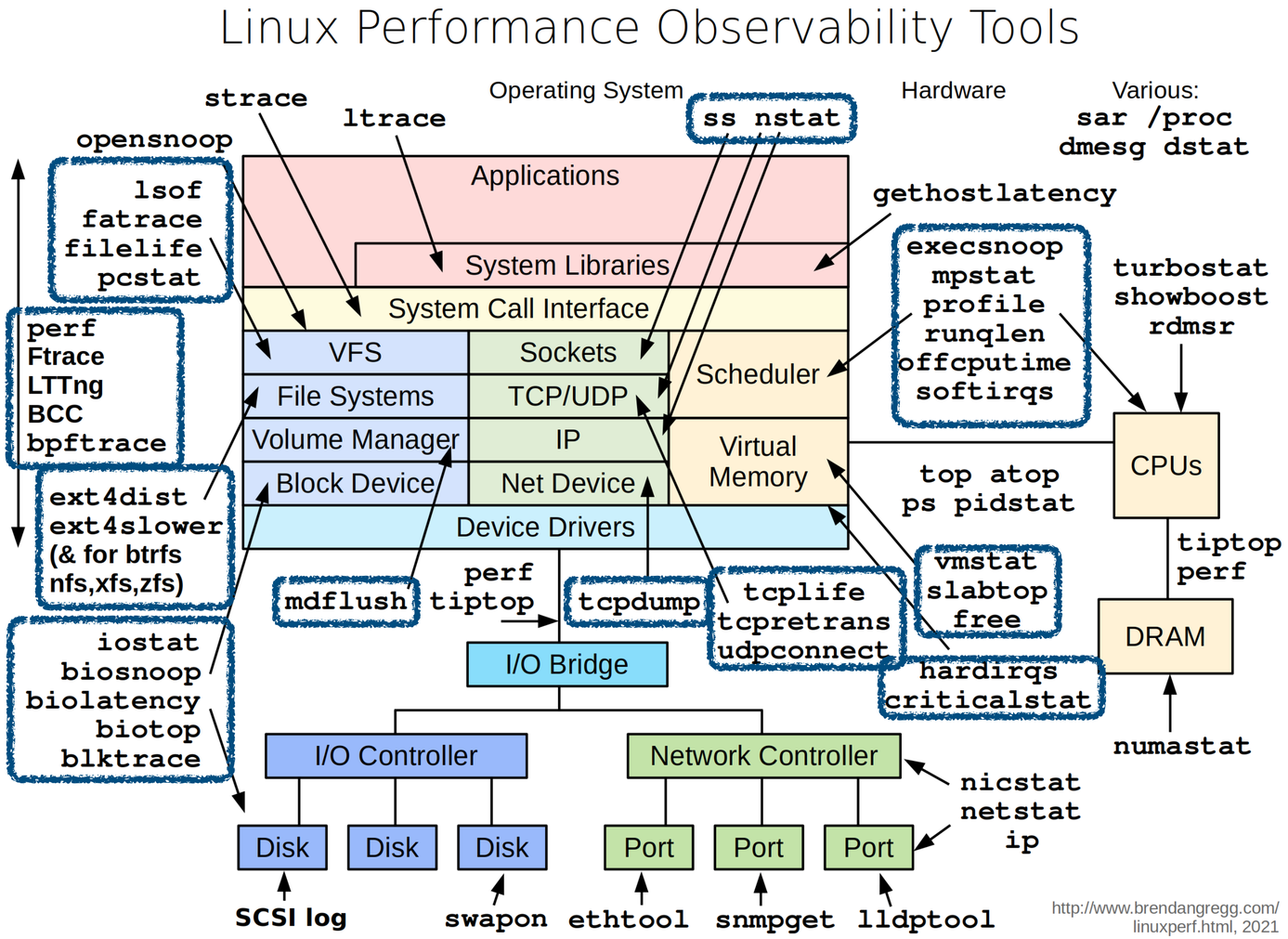
- At the **system call interface**
  - Captures interactions between user space and the operating system.
  - Example: open(), read(), write(), fork(), exec().
  - Tools: strace (system call tracing), …



Data Collection

Where to collect

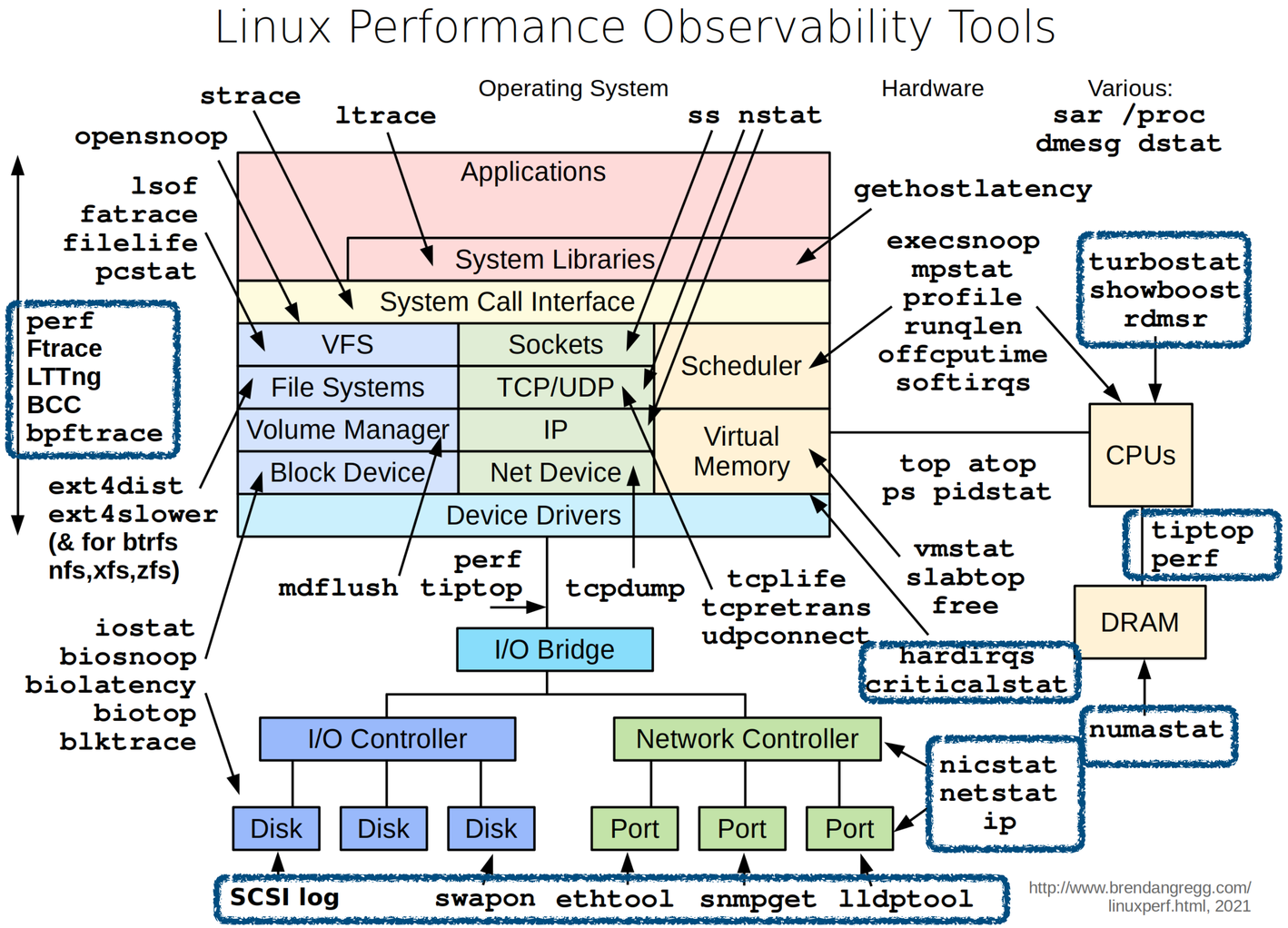
- At the **Linux kernel** level
  - Captures internal OS mechanisms and resource management.
  - Example: VFS, TCP/IP stack, scheduler, virtual memory.
  - Tools: iostat (I/O statistics), netstat (network stats), vmstat (virtual memory stats), …



Data Collection

Where to collect

- At the **Hardware** level
  - Captures low-level physical resource behavior and performance signals.
  - Example: CPU performance counters (cache misses, branch mispredictions).
  - Tools: perf (CPU performance counters, hardware events), …



Data Collection

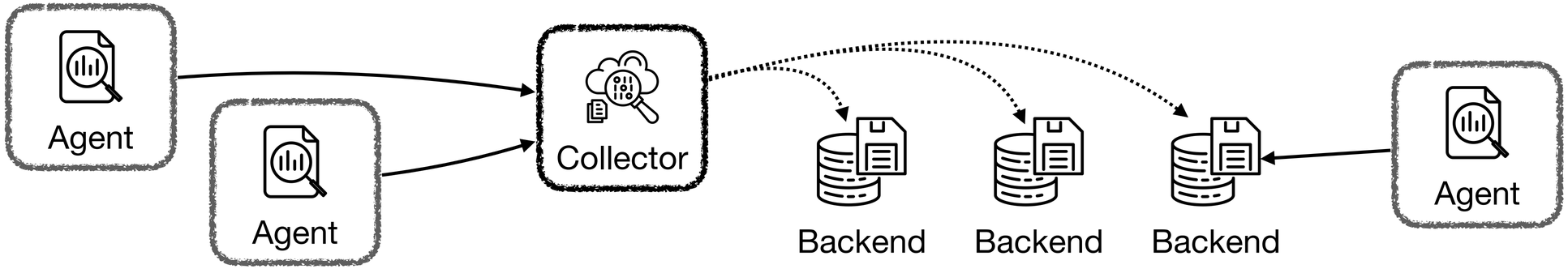
How to collect

- **Instrumentation** — widely used technique for observability.
  - Insertion of code into a system or application to collect observability data.
- **Data sources** (Instrumentation points)
  - **Tracepoints/USDT**[1]: Predefined, stable, static hooks in code.
    - Examples: Kernel tracepoints (syscalls), USDT (Java GC events, PostgreSQL queries).
  - **Probes**: Runtime-injected instrumentation at arbitrary code locations without recompilation or restarts.
    - Examples: kprobes/kretprobes (kernel functions), uprobes/uretprobes (userspace functions).

[1] User Statically-Defined Tracing (USDT)

Data Transport

- Once collected, data moves from sources to storage or analysis tools.
- **Agents**: Collect data and forward it to a collector or directly to a backend.
  - Run locally, close to the source.
  - Example: Fluentd agent, OpenTelemetry Exporter, Metricbeat.
- **Collectors**: Aggregates, processes, and routes data from multiple agents to multiple backends.
  - Can perform batching, filtering, sampling, enrichment, and transformation.
  - Example: Fluent Bit, OpenTelemetry Collector, Logstash.



Data Storage

- Observability data may then be stored in specialized backends for querying and analysis.
- Different backends according to the data type:
  - **Metrics Backends**: Optimized for time-series numeric data.
    - Efficient for aggregation, dashboards, and alerting.
    - Examples: Prometheus, InfluxDB, Graphite.
  - **Logs Backends**: Optimized for indexed, searchable event data.
    - Designed for full-text search and structured queries.
    - Examples: Elasticsearch, Loki, Splunk.
  - **Traces Backends**: Optimized for request/transaction lifecycle data.
    - Support operation flow indexing and trace correlation.
    - Examples: Jaeger, Zipkin, Tempo.

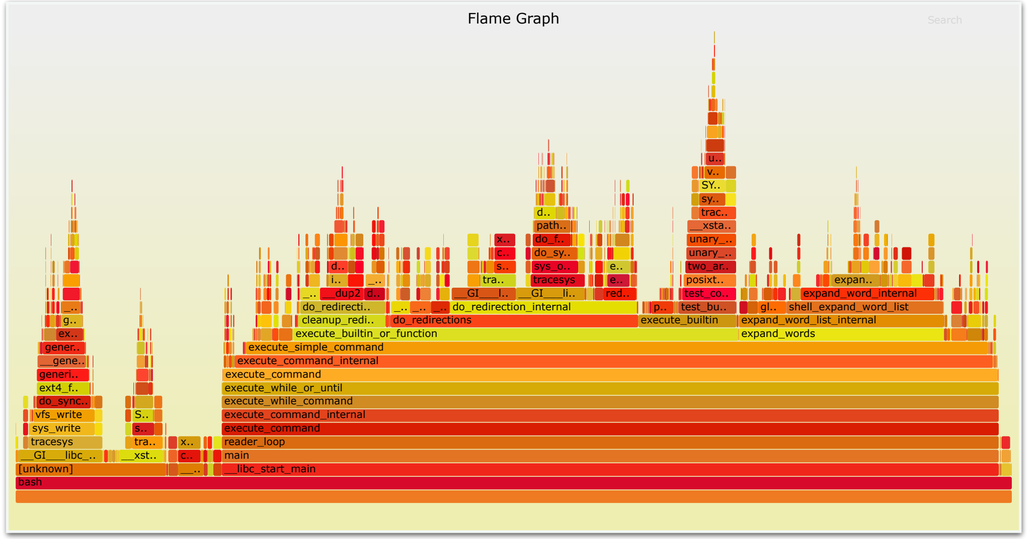
Data Analysis & Visualization

- Finally, observability data can be analyzed and visualized to extract actionable insights.
- **Data Analysis:** Correlate metrics, logs, and traces to identify root causes, detect anomalies, bottlenecks, and failures.
  - Examples:
    - Linking a latency spike (metrics) to an error message (logs).
    - Following a slow request across services (trace correlation).
- **Data visualization:** Designing dashboards or visual interfaces to help humans understand and act on live and historical observability data.
  - Improves cognition, pattern recognition, and decision-making.
  - Examples: Grafana, Kibana, Jaeger UI.

Data Analysis & Visualization

Common visualizations

- **Time-series graphs** (CPU, latency, error rate)
- **Heatmaps** and **histograms** (latency distributions)
- **Flame graphs** (CPU profiling, hotspots)
- **Control flow graphs** (function relationships)
- **Trace timelines** (request spans[1])



Flame graph

[1] Spans: a logical unit of work that has an operation name, the start time of the operation, and the duration.

Some considerations

Challenges

- Collecting data too frequently or with too much detail increases resource usage and performance overhead.
- Sending large amounts of data can congest networks.
- Storing large amounts of data for long periods is expensive and can slow queries.
- Analyzing detailed data across metrics, logs, and traces can be costly and complex.

Some considerations

Techniques

- **Filtering** by certain types of data (e.g., consider only error logs).
- **Sampling** representative subsets of data (e.g., consider 1 in every 100 write operations).
- **Aggregating** data (e.g., summing request counts or averaging latency).
- **Summarizing** data (e.g., min/max values, average/standard deviation, histograms).
- **Batching** requests (e.g., group several data requests over the network).
- Setting **retention policies** to define how long data is stored (e.g., 7 days, 3 months, 1 year).

Disclaimer: Most of these **trade off** accuracy and visibility for better performance and resource usage.

More Information

- Brendan Gregg. **BPF Performance Tools**. Addison-Wesley, 2020.
- Andy Petrella. **Fundamentals of Data Observability: Implement Trustworthy End-to-End Data Solutions**. O'Reilly, 2023.
- Anish Athalye, Jon Gjengset and Jose Ortiz. **Debugging and Profiling.** https://missing.csail.mit.edu/2020/debugging-profiling/

Questions?